ONNX and Quantization

Daily Study Plan: 30 minutes a day, Week 1 Day 1 to Week 6 Day 3

How to use this plan

Each day has one topic. Split your 30 minutes like this: **10 min** read the Concept, **15 min** do the Task, **5 min** write results in your log. Day 6 of every week is review and practice, so you can catch up if you missed a day. Do this after your Brototype tasks; your main curriculum and Paper 3 review come first. The plan runs 5 weeks plus a 3-day capstone, about 6 weeks in total.

All topics at a glance

Topic	Module	Day
Why inference optimization matters	Module 1: Foundations	Week 1, Day 1
Training vs. inference	Module 1: Foundations	Week 1, Day 2
Number formats: FP32, FP16, BF16, INT8	Module 1: Foundations	Week 1, Day 3
Profiling basics	Module 1: Foundations	Week 1, Day 4
What ONNX is	Module 2: ONNX Basics	Week 1, Day 5
Week 1 review and practice	Review	Week 1, Day 6
Exporting from PyTorch	Module 2: ONNX Basics	Week 2, Day 1
Exporting from scikit-learn	Module 2: ONNX Basics	Week 2, Day 2
Inspecting a model with Netron	Module 2: ONNX Basics	Week 2, Day 3
Validating the export	Module 2: ONNX Basics	Week 2, Day 4
Inference sessions	Module 3: ONNX Runtime	Week 2, Day 5
Week 2 review and practice	Review	Week 2, Day 6
Benchmarking against PyTorch	Module 3: ONNX Runtime	Week 3, Day 1
Graph optimizations	Module 3: ONNX Runtime	Week 3, Day 2
Common export errors	Module 3: ONNX Runtime	Week 3, Day 3
What quantization is	Module 4: Quantization Theory	Week 3, Day 4
Symmetric vs. asymmetric, per-tensor vs. per-channel	Module 4: Quantization Theory	Week 3, Day 5
Week 3 review and practice	Review	Week 3, Day 6
Dynamic vs. static vs. QAT	Module 4: Quantization Theory	Week 4, Day 1
Calibration data	Module 4: Quantization Theory	Week 4, Day 2
Where accuracy loss comes from	Module 4: Quantization Theory	Week 4, Day 3
ONNX Runtime dynamic quantization	Module 5: Quantization in Practice	Week 4, Day 4
ONNX Runtime static quantization	Module 5: Quantization in Practice	Week 4, Day 5
Week 4 review and practice	Review	Week 4, Day 6
PyTorch quantization (brief) and measuring the trade-off	Module 5: Quantization in Practice	Week 5, Day 1
Transformer-specific notes	Module 5: Quantization in Practice	Week 5, Day 2
LLM quantization	Module 6: Advanced	Week 5, Day 3
TensorRT, OpenVINO, TFLite, pruning, distillation	Module 6: Advanced	Week 5, Day 4
Serving behind FastAPI	Module 6: Advanced	Week 5, Day 5
Capstone setup	Capstone	Week 5, Day 6

Topic	Module	Day
Run all variants and fill the table	Capstone	Week 6, Day 1
Write the README	Capstone	Week 6, Day 2
Publish and prepare to talk about it	Capstone	Week 6, Day 3

Your log (keep one file all month)

Create `optimization_log.md` on Day 1. After every task, add the date, what you did and the numbers you got. These notes become your capstone README.

WEEK 1: Foundations, then ONNX Basics begins

Why inference optimization matters - WEEK 1, DAY 1 Module 1: Foundations	
TOPICS COVERED	- Latency (time per request), throughput (requests per second), and the tradeoff between them - Model size on disk vs. memory at runtime - Cost: cloud bills, edge devices, battery - Where the time goes: compute-bound vs. memory-bound
CONCEPT	Training teaches the model. **Inference** is using the trained model to predict. **Latency** is the time for one request (for example 80 ms). **Throughput** is how many requests per second the system handles. They pull against each other: batching raises throughput but makes each request wait longer. **Model size on disk** is the file; **memory at runtime** is bigger because activations also need space. A model is **compute-bound** when math is the bottleneck and **memory-bound** when moving weights from RAM is the bottleneck. At batch size 1 most models are memory-bound, which is exactly why smaller weights (quantization) make them faster.
TASK (about 15 min)	Pick one model you already built. Write down its file size. Time one prediction with `time.perf_counter()`. Save both numbers in a file called `optimization_log.md`.
DONE WHEN	You can explain latency vs. throughput in two sentences without looking.

Training vs. inference - WEEK 1, DAY 2 Module 1: Foundations	
TOPICS COVERED	- Forward pass only: no gradients, no optimizer state - model.eval(), torch.no_grad(), torch.inference_mode() - Batch size differences (training uses large batches, serving often uses 1) - Why training code is rarely production-ready
CONCEPT	Training = forward pass + backward pass (gradients) + optimizer update. Inference = forward pass only, so no gradients and no optimizer state are needed, which saves memory. In PyTorch, `model.eval()` switches layers such as Dropout and BatchNorm to their inference behaviour. `torch.no_grad()` and `torch.inference_mode()` stop gradient tracking. Forgetting `eval()` gives noisy, wrong outputs. Training uses large batches; serving often uses batch size 1. That is why training code is rarely production-ready as it is.
TASK (about 15 min)	Run the same input through your PyTorch model three ways: (a) default, (b) with `eval()`, (c) with `eval()` and `torch.inference_mode()`. Compare outputs and time. Add results to the log.
DONE WHEN	You know what `eval()` changes and why outputs differ without it.

Number formats: FP32, FP16, BF16, INT8 - WEEK 1, DAY 3 Module 1: Foundations	
TOPICS COVERED	- FP32 structure: sign, exponent, mantissa - FP16 vs. BF16: range vs. precision - INT8 and INT4 basics - Overflow, underflow, and rounding error
CONCEPT	A float has three parts: sign, **exponent** (controls range), **mantissa** (controls precision). **FP32**: 1 + 8 + 23 bits. **FP16**: 1 + 5 + 10 bits, small range, so values above about 65,504 overflow. **BF16**: 1 + 8 + 7 bits, same range as FP32 but less precision, which is why it is popular for deep learning. **INT8**: whole numbers from -128 to 127, 4 times smaller than FP32. **Rounding error**: every value is snapped to the nearest number the format can store, and many tiny errors can add up.
TASK (about 15 min)	In Python with numpy, convert 0.1 and 70000.0 to `float16` and `float32` and print them. Print `np.finfo(np.float16)` and `np.finfo(np.float32)`. Note what overflows.
DONE WHEN	You can say why BF16 is popular and what INT8 gives up.

Profiling basics - WEEK 1, DAY 4 Module 1: Foundations	
TOPICS COVERED	- Warm-up runs and why the first call is slow - Averaging over many runs, p50/p95/p99 latency - Effect of batch size and input length - Tools: time.perf_counter, torch.profiler, timeit
CONCEPT	The first call is always slow (lazy setup, memory allocation), so do **warm-up runs** and ignore them. Measure many runs, never one. Report **p50** (typical request), **p95** and **p99** (the slow requests users actually notice). Batch size and input length change latency, so always state them. Tools: `time.perf_counter` (accurate clock), `timeit`, `torch.profiler` (time per operator).
TASK (about 15 min)	Write a function `benchmark(fn, warmup=10, runs=100)` that returns mean, p50 and p95 in milliseconds. Run it on your model at batch sizes 1, 8, 32. Save it as `benchmark.py`. You will reuse it all month.
DONE WHEN	`benchmark.py` works and you have a latency table for 3 batch sizes.

What ONNX is - WEEK 1, DAY 5 Module 2: ONNX Basics	
TOPICS COVERED	- Open format for interchange between frameworks - Computation graph: nodes, edges, initializers (weights) - Operators and opsets (version compatibility) - ONNX vs. ONNX Runtime (format vs. engine)
CONCEPT	ONNX is a file format that describes a model as a **graph**. **Nodes** are operators (MatMul, Relu). **Edges** are the tensors flowing between nodes. **Initializers** are the stored weights. An **opset** is the version of the operator definitions; the exporter and the runtime must agree on it. ONNX is only the **format**. **ONNX Runtime** is the **engine** that runs it. The benefit: train in PyTorch, then run anywhere without needing PyTorch installed.
TASK (about 15 min)	Install the tools: `pip install onnx onnxruntime netron`. Read the intro page on onnx.ai for 10 minutes. Then write 5 lines in your own words: node, edge, initializer, opset, runtime.
DONE WHEN	You can explain ONNX vs. ONNX Runtime to a friend.

Week 1 review and practice - WEEK 1, DAY 6 Review	
CONCEPT	No new concept today. Revise: latency vs. throughput, eval() vs. train mode, FP32/FP16/BF16/INT8, warm-up and percentiles, ONNX vs. ONNX Runtime.
TASK (about 15 min)	Re-run `benchmark.py`. Write a one-page 'Week 1 notes' file. Quiz yourself: (1) why warm-up? (2) what does BF16 trade? (3) what is an opset? (4) what is memory-bound? (5) ONNX or ONNX Runtime: which one executes the model?
DONE WHEN	You answered all 5 questions without notes.

WEEK 2: ONNX Basics and ONNX Runtime

Exporting from PyTorch - WEEK 2, DAY 1 Module 2: ONNX Basics	
TOPICS COVERED	- torch.onnx.export arguments: dummy input, opset_version - Naming inputs and outputs - Dynamic axes for variable batch size and sequence length - Tracing vs. scripting, and why control flow can break exports
CONCEPT	`torch.onnx.export` runs your model once on a **dummy input** and records the operations (**tracing**). So the dummy input must have the right shape and dtype. Give `input_names` and `output_names` so the file is readable. `dynamic_axes` marks dimensions that may change (batch size, sequence length); without it, the batch size is frozen to the dummy's. Pick a recent `opset_version`. Warning: data-dependent if/for logic is not captured by tracing and can break or silently freeze.
TASK (about 15 min)	Export your PyTorch model to `model.onnx` with `input_names=['input']`, `output_names=['output']` and a dynamic batch axis. Note the file size in your log.
DONE WHEN	`model.onnx` exists (batch-size test happens on Day 4).

Exporting from scikit-learn - WEEK 2, DAY 2 Module 2: ONNX Basics	
TOPICS COVERED	- skl2onnx and convert_sklearn - Declaring input types (FloatTensorType) - Pipelines (scaler + model) in one ONNX file - Supported vs. unsupported estimators
CONCEPT	`skl2onnx` converts scikit-learn models. You call `convert_sklearn(model, initial_types=[('input', FloatTensorType([None, n_features]))])`. **None** means any batch size. A scikit-learn **Pipeline** (scaler + model) converts as one unit, so preprocessing lives inside the ONNX file. Not every estimator is supported, so check the supported list when a conversion fails.
TASK (about 15 min)	Train a small Pipeline (StandardScaler + LogisticRegression or RandomForest) on any dataset from your course. Convert it and save `sk_model.onnx`.
DONE WHEN	`sk_model.onnx` is saved and the conversion ran without errors.

Inspecting a model with Netron - WEEK 2, DAY 3 Module 2: ONNX Basics	
TOPICS COVERED	- Reading the graph visually - Checking input/output shapes and dtypes - Spotting unexpected or unsupported ops
CONCEPT	Netron draws the ONNX graph so you can see what the exporter really produced. Check: input and output names, shapes, dtypes; the order of operators; node attributes (click a node). Many Shape, Gather or Cast nodes usually mean dynamic-shape handling. Unexpected or unsupported operators show up here first.
TASK (about 15 min)	Open both ONNX files in Netron (`netron model.onnx` or netron.app). Take a screenshot of each. List all operator types you can see.
DONE WHEN	You can name the first 3 and last 3 operators of your model.

Validating the export - WEEK 2, DAY 4 Module 2: ONNX Basics	
TOPICS COVERED	- onnx.load and onnx.checker.check_model - Shape inference - Comparing ONNX vs. original outputs with np.allclose - Choosing a sensible tolerance (rtol/atol)
CONCEPT	`onnx.load` and `onnx.checker.check_model` confirm the file is structurally valid. Shape inference fills in tensor shapes. But a valid file can still give wrong numbers, so always compare against the original model with `np.allclose(a, b, rtol=1e-3, atol=1e-5)`. Small differences are normal because operations may be ordered differently.
TASK (about 15 min)	Feed the same 100 inputs to PyTorch and to ONNX Runtime. Compute the max absolute difference. Run `allclose`. Repeat for batch sizes 1 and 16.
DONE WHEN	You have a reported max difference, under tolerance, for both batch sizes.

Inference sessions - WEEK 2, DAY 5 Module 3: ONNX Runtime	
TOPICS COVERED	- Creating an InferenceSession - Providers: CPUExecutionProvider and others - session.run, input dict format, numpy dtypes - Session options: thread counts, execution mode
CONCEPT	`ort.InferenceSession('model.onnx', providers=['CPUExecutionProvider'])` loads the model. `session.run(None, {'input': x})` runs it; **x** must be a numpy array with the right dtype. A **provider** is the hardware backend; CPU is the default and enough for this course. `SessionOptions` lets you set thread counts and execution mode. Classic mistake: passing float64 arrays to a float32 model.
TASK (about 15 min)	Write `predict_onnx(x)`. Print `session.get_inputs()` (name, shape, type). Run 5 samples.
DONE WHEN	ONNX predictions match PyTorch on 5 samples.

Week 2 review and practice - WEEK 2, DAY 6 Review	
CONCEPT	No new concept. Revise: export arguments, dynamic axes, skl2onnx, Netron, validation, sessions.
TASK (about 15 min)	Export a second model end to end **without notes**: export, check, Netron, validate, run in ONNX Runtime. Time yourself and write down what you looked up.
DONE WHEN	The full pipeline works from memory, or you have a list of the gaps.

WEEK 3: Benchmarking, Optimizations, and Quantization Theory

Benchmarking against PyTorch - WEEK 3, DAY 1 Module 3: ONNX Runtime	
TOPICS COVERED	- Fair comparison setup (same input, warm-up, same threads) - Single-sample vs. batched latency - Model load time vs. inference time - Recording results in a table
CONCEPT	A fair comparison needs the same input, the same warm-up, the same batch sizes and the **same thread count** (`torch.set_num_threads` and ORT `intra_op_num_threads`). Measure single-sample and batched latency separately. **Model load time** is not inference time; report it on its own.
TASK (about 15 min)	Use `benchmark.py` to build a table: PyTorch FP32 vs. ONNX FP32, batch 1, 8, 32, with mean and p95.
DONE WHEN	The table is in your log with one sentence: which is faster and by how much.

Graph optimizations - WEEK 3, DAY 2 Module 3: ONNX Runtime	
TOPICS COVERED	- Optimization levels (basic, extended, all) - Operator fusion (e.g., Conv+BN+ReLU, attention fusion) - Constant folding and redundant node removal - Saving the optimized model
CONCEPT	ORT optimization levels: **basic** (constant folding, removing redundant nodes), **extended** (operator fusion, e.g. Conv + BatchNorm + ReLU becomes one node), **all** (hardware-specific layout changes). Fusion means fewer trips to memory, so it runs faster. Set `sess_options.graph_optimization_level` and `sess_options.optimized_model_filepath` to save the optimized graph.
TASK (about 15 min)	Run the model at levels: disabled, basic, extended, all. Benchmark each. Open the saved optimized model in Netron and compare node counts.
DONE WHEN	A small table: level, node count, latency.

Common export errors - WEEK 3, DAY 3 Module 3: ONNX Runtime	
TOPICS COVERED	- Unsupported operators and opset fixes - Shape mismatch and dynamic axes mistakes - dtype mismatches (float64 vs. float32, int32 vs. int64) - Debugging approach: export a smaller piece, bisect
CONCEPT	**Unsupported operator**: try a higher opset or rewrite that part of the model. **Shape mismatch**: usually `dynamic_axes` is missing or wrong. **dtype mismatch**: float64 vs. float32, int32 vs. int64 (token ids are normally int64). Debug method: export a smaller piece (one layer), then **bisect** until the failing part is found.
TASK (about 15 min)	Break things on purpose: export without dynamic axes and run batch 4; feed float64 input. Read each error message and write what it means and the fix.
DONE WHEN	3 errors documented, each with cause and fix.

What quantization is - WEEK 3, DAY 4 Module 4: Quantization Theory	
TOPICS COVERED	- Mapping floats to integers: q = round(x / scale) + zero_point - Dequantization and rounding error - What actually gets quantized: weights, activations, or both - Benefits: smaller size, faster integer math, lower memory bandwidth
CONCEPT	Store numbers as 8-bit integers instead of 32-bit floats. Formula: `q = round(x / scale) + zero_point`. To get back: `x_approx = (q - zero_point) x scale`. The gap between `x` and `x_approx` is the **rounding error**. Weights, activations, or both can be quantized. Benefits: about 4 times smaller, faster integer math on CPUs that support it, less memory traffic.
TASK (about 15 min)	In numpy: take 10 floats. Compute `scale = (max - min) / 255`. Quantize to uint8, dequantize, print the error for each value.
DONE WHEN	You can quantize and dequantize by hand and compute the error.

Symmetric vs. asymmetric, per-tensor vs. per-channel - WEEK 3, DAY 5 Module 4: Quantization Theory	
TOPICS COVERED	- Zero-point = 0 (symmetric) vs. non-zero (asymmetric) - When asymmetric helps (e.g., ReLU outputs) - Per-channel scales for convolution and linear weights - Tradeoff: accuracy vs. complexity
CONCEPT	**Symmetric**: zero_point = 0, range is -max to +max. Simple and fast; good for weights. **Asymmetric**: zero_point is not 0, range fits min to max exactly; good for activations after ReLU (all values are 0 or more). **Per-tensor**: one scale for the whole tensor. **Per-channel**: one scale per output channel, which is more accurate when channels have very different ranges. Trade-off: accuracy vs. complexity.
TASK (about 15 min)	In numpy, build a weight matrix whose rows have very different ranges. Quantize it per-tensor and per-channel. Compare the mean error.
DONE WHEN	Per-channel shows lower error and you can explain why.

Week 3 review and practice - WEEK 3, DAY 6 Review	
CONCEPT	No new concept. Revise: fair benchmarking, optimization levels, export errors, the quantization formula, symmetric/asymmetric, per-tensor/per-channel.
TASK (about 15 min)	Write the quantization formula and a worked example from memory. Then re-explain the Day 5 experiment in your own words in 5 lines.
DONE WHEN	You can derive scale and zero_point for a given min and max.

WEEK 4: Quantization Theory (finish) and Quantization in Practice

Dynamic vs. static vs. QAT - WEEK 4, DAY 1 Module 4: Quantization Theory	
TOPICS COVERED	- Dynamic: weights quantized offline, activations at runtime - Static: both quantized offline using calibration - QAT: simulating quantization during training (fake-quant nodes) - Choosing between them
CONCEPT	**Dynamic**: weights are quantized in advance; activation scales are computed on the fly at runtime. Easiest, needs no data. **Static**: weights and activations are both quantized in advance using **calibration data**. Faster, but needs data. **QAT** (quantization-aware training): fake-quantize nodes are added during training so the model learns to tolerate rounding. Best accuracy, most work. Rule of thumb: try dynamic first, then static, and QAT only if accuracy drops too much.
TASK (about 15 min)	Draw a table with rows Dynamic, Static, QAT and columns: needs data, effort, speed, accuracy. Fill it from memory, then check against the ONNX Runtime docs.
DONE WHEN	Given a scenario, you can choose the method and defend it.

Calibration data - WEEK 4, DAY 2 Module 4: Quantization Theory	
TOPICS COVERED	- Why activation ranges must be estimated - Representative sample size (often 100-500 samples) - Calibration methods: MinMax, Entropy, Percentile - Risks of unrepresentative calibration data
CONCEPT	Static quantization must know the typical range of every activation, so you run a few hundred real samples through the model and record the ranges. Use **100 to 500 representative samples** from the same distribution as real inputs. Methods: **MinMax** (uses extremes, hurt by outliers), **Entropy** (clips outliers by minimizing information loss), **Percentile** (ignores the extreme x%). Unrepresentative calibration data leads to bad ranges and lost accuracy.
TASK (about 15 min)	In numpy: make an array with a few huge outliers. Compute the MinMax range and the 99.9th-percentile range. Quantize with both. Compare error on the normal values.
DONE WHEN	You can explain why outliers ruin the MinMax method.

Where accuracy loss comes from - WEEK 4, DAY 3 Module 4: Quantization Theory	
TOPICS COVERED	- Outliers stretching the range - Sensitive layers (first/last layers, attention, softmax) - Mixed precision: leaving some layers in FP32 - Measuring loss per layer
CONCEPT	**Outliers** stretch the range, so most values use only a few of the 256 levels. **Sensitive layers**: first and last layers, attention, softmax. Fix: **mixed precision**, keeping sensitive layers in FP32 (exclude those nodes from quantization). To find the culprits, compare FP32 and INT8 outputs layer by layer.
TASK (about 15 min)	Read the ORT quantization docs on `nodes_to_exclude`. Write a short plan: which layers of your model you would exclude, and why.
DONE WHEN	You can name 3 causes of accuracy loss and the fix for each.

ONNX Runtime dynamic quantization - WEEK 4, DAY 4 Module 5: Quantization in Practice	
TOPICS COVERED	- quantize_dynamic with QuantType.QInt8 - Which ops it targets (MatMul, Gemm, LSTM) - Quick before/after size check - Limitations (mostly helps linear-heavy models)
CONCEPT	`from onnxruntime.quantization import quantize_dynamic, QuantType`, then `quantize_dynamic('model.onnx', 'model_int8_dyn.onnx', weight_type=QuantType.QInt8)`. It targets operators such as MatMul, Gemm and LSTM, so it helps most for models heavy on linear layers (MLPs, Transformers) and less for convolution-heavy ones.
TASK (about 15 min)	Quantize your model dynamically. Compare file size, run the benchmark, and measure accuracy on your validation set.
DONE WHEN	Size, latency and accuracy for INT8 dynamic are in your results table.

ONNX Runtime static quantization - WEEK 4, DAY 5 Module 5: Quantization in Practice	
TOPICS COVERED	- Writing a CalibrationDataReader - QDQ format vs. QOperator format - quantize_static options (per-channel, activation type) - Pre-processing step before quantizing
CONCEPT	Write a class that extends `CalibrationDataReader` with `get_next()` returning `{'input': batch}` and `None` when finished. Run `quant_pre_process` first. Formats: **QDQ** (QuantizeLinear/DequantizeLinear pairs, recommended and flexible) vs. **QOperator**. Options to know: `per_channel`, `activation_type`, `weight_type`.
TASK (about 15 min)	Build the reader with 200 samples from your validation set. Call `quantize_static`. Benchmark and measure accuracy.
DONE WHEN	Size, latency and accuracy for INT8 static are in your results table.

Week 4 review and practice - WEEK 4, DAY 6 Review	
CONCEPT	No new concept. Revise: dynamic vs. static vs. QAT, calibration, accuracy-loss causes, both ORT quantization APIs.
TASK (about 15 min)	Compare your dynamic and static results side by side. Write a 5-line conclusion: which won on size, speed and accuracy, and why you think so.
DONE WHEN	You have a written conclusion backed by your numbers.

WEEK 5: Transformers, LLMs, Other Tools, and Serving

PyTorch quantization (brief) and measuring the trade-off - WEEK 5, DAY 1 Module 5: Quantization in Practice	
TOPICS COVERED	- Eager mode vs. FX graph mode - torch.ao.quantization overview - Comparing results with the ONNX approach - Backend notes (fbgemm for x86, qnnpack for ARM) - Measuring the tradeoff: size, latency, accuracy, and building the results table
CONCEPT	`torch.ao.quantization` offers **eager mode** (manual steps) and **FX graph mode** (automatic). Backends: **fbgemm** for x86, **qnnpack** for ARM. Whatever tool you use, always report the same three numbers with the same benchmark harness: **size, latency, accuracy**.
TASK (about 15 min)	Skim the PyTorch quantization overview (10 min only). Try `torch.quantization.quantize_dynamic` on a Linear-based model and compare with ORT. Create the empty results table in your README.
DONE WHEN	The results table skeleton exists and you have one PyTorch-vs-ORT comparison.

Transformer-specific notes - WEEK 5, DAY 2 Module 5: Quantization in Practice	
TOPICS COVERED	- Exporting BERT-style models with Hugging Face Optimum - Why attention and LayerNorm are harder to quantize - Sequence-length and dynamic-axes handling - Expected speedups on CPU
CONCEPT	Hugging Face **Optimum** exports Transformers to ONNX (`optimum-cli export onnx --model NAME out/`). Attention and LayerNorm are harder to quantize because activations have large outliers, and softmax is sensitive. Use dynamic axes for batch size and sequence length. Speedups on CPU vary by model and processor, so measure your own.
TASK (about 15 min)	Export `distilbert-base-uncased-finetuned-sst-2-english` with Optimum. Quantize it dynamically. Compare size and latency on 100 sentences.
DONE WHEN	You have size and latency numbers for FP32 vs. INT8 on a Transformer.

LLM quantization - WEEK 5, DAY 3 Module 6: Advanced	
TOPICS COVERED	- Weight-only quantization (INT8, INT4) - GPTQ and AWQ: what problem each solves - GGUF format and llama.cpp quant levels (Q4_K_M, Q5_K_M, Q8_0) - Memory math: parameters x bits / 8, and what fits in your 16GB RAM / 8GB VRAM
CONCEPT	LLMs mostly use **weight-only quantization** (INT8 or INT4). **GPTQ**: post-training, uses calibration data to reduce layer output error, mostly for GPUs. **AWQ**: protects the small set of weights that matter most, judged by activations. **GGUF**: the file format used by llama.cpp; levels such as **Q4_K_M** (good balance), **Q5_K_M**, **Q8_0** (near lossless). **Memory math**: parameters x bits / 8 = bytes. A 7B model is about 14 GB at FP16 and about 3.5 GB at 4-bit, plus overhead and KV cache.
TASK (about 15 min)	Make a table for 3B, 7B and 13B models at FP16, INT8 and 4-bit. Mark which fit in 8 GB VRAM and which fit in 16 GB RAM.
DONE WHEN	You can estimate in seconds whether a model fits your hardware.

TensorRT, OpenVINO, TFLite, pruning, distillation - WEEK 5, DAY 4 Module 6: Advanced	
TOPICS COVERED	- TensorRT: NVIDIA GPUs, engine building, FP16/INT8 - OpenVINO: Intel CPUs and accelerators - TFLite: mobile and edge - Decision guide: hardware target determines the tool - Unstructured vs. structured pruning - Knowledge distillation (teacher/student, soft labels), e.g. DistilBERT
CONCEPT	**TensorRT**: NVIDIA GPUs, builds an optimized engine, FP16/INT8. **OpenVINO**: Intel CPUs and accelerators. **TFLite**: mobile and edge devices. The hardware decides the tool. **Pruning**: remove weights. Unstructured pruning zeroes single weights and needs sparse support to be faster; structured pruning removes whole channels or heads and gives real speedups. **Distillation**: a small student model learns from a large teacher's soft probabilities. Example: DistilBERT.
TASK (about 15 min)	Write one short paragraph per tool. Then choose the best tool for: an NVIDIA server, an Intel laptop, an Android app. Give one reason each.
DONE WHEN	Three correct tool choices with reasons.

Serving behind FastAPI - WEEK 5, DAY 5 Module 6: Advanced	
TOPICS COVERED	- Load the ONNX session once at startup - Input validation with Pydantic - Batching and concurrency basics - Dockerizing the service (ties into Paper 4) - Basic load testing (e.g., locust or hey)
CONCEPT	Load the ONNX session **once at startup**, never per request. Validate input with Pydantic. Concurrency: uvicorn workers and ORT thread count must not fight over the same cores. Package it with Docker (this connects to Paper 4). Test with a load tool such as `locust` or `hey`.
TASK (about 15 min)	Build `main.py` with a `/predict` endpoint using your quantized ONNX model. Test it with curl. Write a Dockerfile.
DONE WHEN	curl returns a correct prediction from your quantized model.

Capstone setup - WEEK 5, DAY 6 Capstone	
CONCEPT	Choose the one model you will present. Everything from this month feeds into this project.
TASK (about 15 min)	Create a repo folder with: `README.md`, `export.py`, `quantize.py`, `benchmark.py`, `results.md`. Move your earlier scripts in.
DONE WHEN	The folder is organized and each script runs on its own.

WEEK 6: Capstone Project

Run all variants and fill the table - WEEK 6, DAY 1 Capstone	
CONCEPT	Use the identical benchmark harness and the identical validation set for every row so the comparison is honest.
TASK (about 15 min)	Run PyTorch FP32, ONNX FP32, ONNX INT8 dynamic and ONNX INT8 static. Record size, latency (p50 and p95) and accuracy for each.
DONE WHEN	Every cell in the results table is filled.

Write the README - WEEK 6, DAY 2 Capstone	
CONCEPT	A good README answers: what problem, what method, what results, what you learned, what the limits are.
TASK (about 15 min)	Write sections: Goal, Setup, Method (export, then quantize), Results table, Findings (what got faster, what accuracy dropped), Limitations, How to run.
DONE WHEN	A stranger could reproduce your results from the README.

Publish and prepare to talk about it - WEEK 6, DAY 3 Capstone	
CONCEPT	Interviewers care about numbers and honest trade-offs, not buzzwords.
TASK (about 15 min)	Push the repo to GitHub. Add one line to your CV. Practise a 60-second explanation: why quantize, your numbers, what you lost, what you would try next.
DONE WHEN	Repo is public and you can present it in 60 seconds.

Capstone results table (fill this in)

Use the same input data, the same benchmark function and the same validation set for every row.

Version	Size (MB)	Latency p50 (ms)	Latency p95 (ms)	Accuracy
PyTorch FP32				
ONNX FP32				
ONNX INT8 dynamic				
ONNX INT8 static				

60-second interview explanation (checklist)

[ ] Why I optimized this model (the problem: latency, size, or memory).

[ ] What I did: exported to ONNX, ran in ONNX Runtime, applied INT8 quantization (dynamic and static).

[ ] My numbers: size, latency and accuracy, before and after.

[ ] What I lost: the accuracy drop, and where I think it came from.

[ ] What I would try next: mixed precision, QAT, or a different hardware target.